Exception Handling

1. Division Calculator

Write a program to divide two numbers. If the user enters zero as the denominator, throw and handle an `ArithmeticException` with a meaningful error message.

---

2. File Reader

Create a program to read the contents of a file. Handle exceptions like `FileNotFoundException` and `IOException`. If the file is not found, prompt the user to provide a valid file path.

---

3. Age Validator

Implement a method `validateAge(int age)` that throws a custom exception `InvalidAgeException` if the age is less than 18. Handle this exception in the calling code and print an appropriate message.

---

4. Array Element Fetcher

Write a program to retrieve an element at a specific index from an array. If the user enters an index out of bounds, handle the `ArrayIndexOutOfBoundsException` and prompt the user to re-enter a valid index.

---

5. User Authentication

Create a program that takes a username and password as input. If the username or password is null, throw a custom `AuthenticationException`. Use `try-catch` to handle this and display a proper message to the user.

---

6. Number Conversion

Write a program that converts a string to an integer using `Integer.parseInt()`. If the input string is invalid (non-numeric), handle the `NumberFormatException` and prompt the user to enter a valid number.

---

7. Resource Management

Develop a program to write data to a file using `FileWriter`. Ensure that the file resource is properly closed by using a `finally` block or a try-with-resources statement.

---

8. Bank Account Management

Simulate a simple banking system:

- Implement a `withdraw(double amount)` method.
- If the withdrawal amount exceeds the account balance, throw a custom `InsufficientFundsException`.
- Handle the exception in the main program and display a user-friendly error message.

---

9. Nested Exceptions

Write a program where one exception causes another exception. For instance:

- Try to access a file that doesn't exist (causing a `FileNotFoundException`).
- Within the `catch` block, attempt to divide by zero (causing an `ArithmeticException`). Use nested `try-catch` blocks to handle both exceptions.

---

10. User Registration

Create a user registration program:

- Validate the user's email format using regular expressions.
- Throw a custom `InvalidEmailException` if the format is incorrect.
- Use a `try-catch` block to handle this exception and ask the user to re-enter a valid email.